

实验二：在线评测系统实验报告

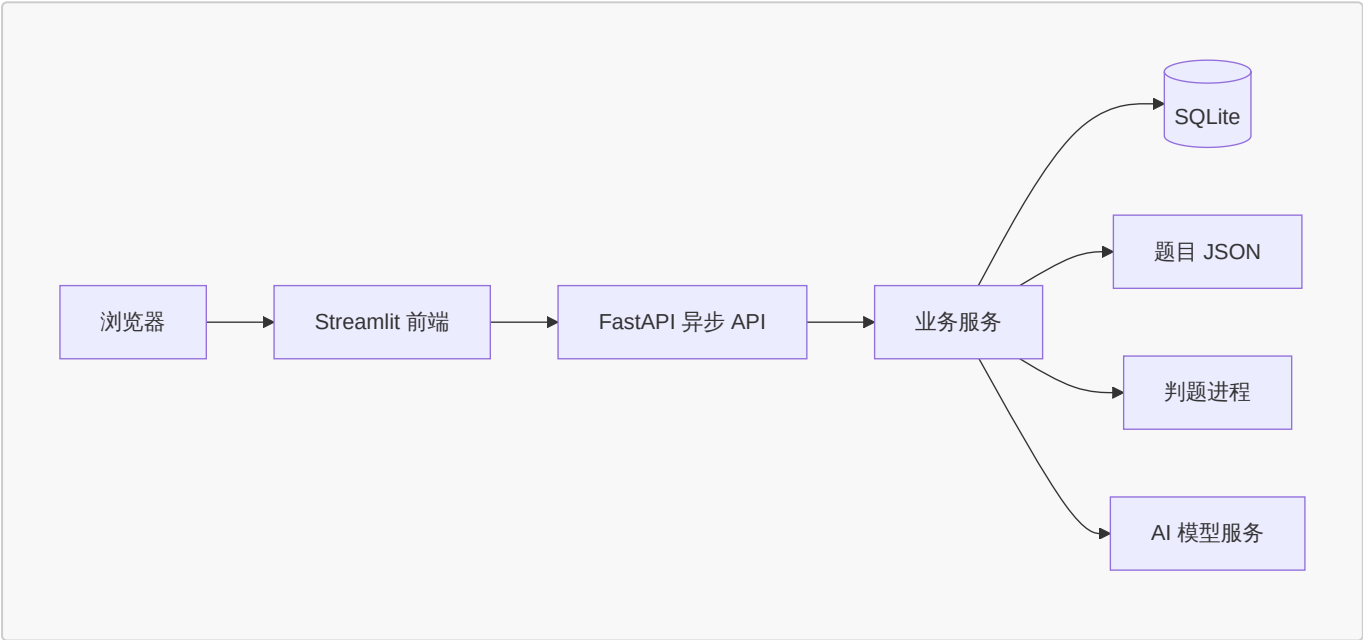
姓名：张家睿 学号：2025010428 提交日期：2026-09-09

1. 实验目标

本实验实现一个可运行的 Online Judge (OJ) 系统。系统以 FastAPI 提供异步 REST API，以 Streamlit 提供浏览器操作界面；完成题目管理、程序评测、用户与权限、评测记录、访问审计和 AI 辅助命题等功能，并保证接口响应、会话状态和权限控制一致。

2. 系统设计

系统采用“前端页面—API—业务服务—存储/执行器”的分层结构。Streamlit 只负责展示和调用 API；FastAPI 路由负责请求和响应；service 层实现业务规则；SQLite 保存用户、会话、语言、提交、日志和 AI 任务，题目按 JSON 文件分别保存在 `data/problems/`。



这样划分的优点是：前后端职责明确，权限判断集中在后端；题库数据符合实验要求的文件存储方式；判题和 AI 调用等耗时任务不会阻塞正常的 API 请求。

3. 功能实现

模块	实现说明
题目管理	支持题目列表、详情、新建、编辑和删除；题目包含样例、测试点、时间/内存限制及日志公开策略。
在线评测	支持 Python 3 与 C++14，并允许登录用户注册语言配置；评测分别处理编译错误、答案错误、运行错误、超时和内存超限。
提交管理	提交后立即返回 pending，后台逐测试点评测并记录分数、状态和编译信息；支持筛选、分页、查看详情和管理员重判。

模块	实现说明
用户与权限	支持注册、登录、登出、修改用户名、用户管理和角色控制；会话使用 HttpOnly Cookie，刷新及浏览器前后退后可恢复登录状态。
访问审计	记录允许及拒绝的敏感访问，列表显示用户名，管理员可在每条日志末尾删除记录。
AI 命题	支持配置 OpenAI 兼容模型、创建任务、查看进度、中断、失败重试、费用与 Token 展示，以及基于生成结果继续对话修改。

3.1 判题与资源控制

提交代码在独立临时目录中编译和运行。系统按“题目限制—语言限制—系统默认值”的优先级确定资源限制，并结合 CPU 时间、墙钟时间、输出大小、进程数和进程树内存监控判断结果。超时、取消和正常结束都会回收进程组及临时目录，避免残留子进程影响后续评测。

输出比较忽略行尾空白和末尾空行。每个通过的测试点计 10 分；正常完成的评测即使答案错误也记为 **success**，只有编译或基础设施异常记为 **error**，从而能正确展示部分得分。

3.2 会话、路由与权限

登录成功后，后端签发不透明会话令牌并设置 HttpOnly Cookie；前端每次加载通过 `/api/auth/me` 校验会话。页面导航统一使用 Streamlit 的 `st.navigation`、`st.page_link` 和 `st.switch_page`，不再创建新浏览器标签或维护自定义历史记录，因此浏览器刷新、前进、后退和页面内返回均保持正确的单页体验。

权限判断在后端完成，普通用户不能修改或删除已有题目，管理员拥有题目管理、重判、用户管理和审计删除权限。接口在解析请求体前优先完成认证和角色检查，保证未登录、无权限与参数错误能返回正确的状态码。

3.3 AI 命题 workflow

AI 配置中的密钥经 Fernet 加密保存，查询接口不返回密钥。创建任务时会固化该次使用的模型配置；后端在后台调用模型、校验其 JSON 输出并记录任务进度。前端提供极速、均衡和高质量三档模式：极速使用较快模型并关闭思考，均衡降低推理强度，高质量在初稿后额外核对答案、边界和复杂度。

生成结果可先审阅测试点和 JSON，再保存为题目；若“保存为新题目”的题号已存在，服务端会安全分配新的题号。对结果不满意时，用户可继续输入修改意见；每轮修改都生成一个新版本，并保留历史版本、用量和费用。失败任务可使用当前配置重新开始，原失败记录不会被覆盖。

4. 验证与结果

开发阶段使用隔离的临时数据库和题库进行 API、权限、判题和前端交互回归验证，覆盖 Python/C++ 编译执行、AC/WA/TLE/MLE/RE、分页筛选、重判、审计、会话恢复、AI 任务进度、取消、重试和费用计算等场景。另在浏览器中人工验证了登录、整页刷新、题目详情、AI 详情、浏览器返回、页面内返回和登出流程。

提交包仅保留系统运行所需代码、配置、题库、启动脚本和本报告；开发期自动化测试、压力数据生成脚本和缓存已清理，不影响使用 `./run.sh` 启动和演示系统。

5. 界面展示



Online Judge
LEARN · CODE · GROW

zjr284
管理员

题目
评测记录
语言管理
AI 命题
个人主页
用户管理
访问审计

退出登录

提交 #15 评测完成

题目 1001 · 用户 3 · 语言 cpp · 提交于 2026-09-09 12:19:30

得分
100.0

总分
100

全部测试点通过。

AC × 10 通过

编译信息（编译成功）
编译成功

运行信息
10 test cases finished
查看提交代码

测试点明细

id	result	time	memory
1	AC	0.018	0
2	AC	0.016	0
3	AC	0.016	0
4	AC	0.016	0
5	AC	0.016	0
6	AC	0.017	0

Online Judge
LEARN · CODE · GROW

zjr284
管理员

题目
评测记录
语言管理
AI 命题
个人主页
用户管理
访问审计

退出登录

访问审计

记录所有评测日志查看行为（允许与拒绝）· 仅管理员可见

用户名（筛选）
留空为全部

题目 ID（筛选）
留空为全部

应用筛选

用户名	题目	行为	结果	时间	操作
zjr284	1001	查看日志	允许	2026-09-09 12:42:42	删除
zjr284	1001	查看日志	允许	2026-09-09 12:42:40	删除
zjr284	1001	查看日志	允许	2026-09-09 12:42:39	删除
zjr284	1001	查看日志	允许	2026-09-09 12:42:37	删除
zjr284	1001	查看日志	允许	2026-09-09 12:42:36	删除
zjr284	1001	查看日志	允许	2026-09-09 12:42:34	删除
zjr284	1001	查看日志	允许	2026-09-09 12:42:33	删除
zjr284	1001	查看日志	允许	2026-09-09 12:42:31	删除

Online Judge
LEARN · CODE · GROW

zjr284

管理员

题目

评测记录

语言管理

AI 命题

个人主页

用户管理

访问审计

退出登录

AI 智能命题

配置大模型后，输入命题需求即可自动生成符合题库规范的题目，实时查看进度并可导入题库。
推理模型可能需要数分钟；单次请求最长等待 600 秒（可通过 OJ_AI_REQUEST_TIMEOUT 调整）。

模型配置

新建命题任务

命题需求

例如：出一道考查二分查找的题目，难度中等， $n \leq 10^6$ ，包含边界测试点

Press Ctrl+Enter to submit form

参考/改编题目（可选）

— 新题目 —

生成模式

☐ 极速 | Flash - 关闭思考

☒ 均衡 | Flash - 低推理强度

☐ 高质量 | Pro - 高推理强度

创建命题任务

任务记录

ID	状态	进度	模式	模型	创建时间
#21	完成	100%	均衡 Flash - 低推理强度	deepseek-v4-flash	2026-09-09 12:08:37
#20	失败	17%	均衡 Flash - 低推理强度	deepseek-v4-flash	2026-09-09 12:05:35

Online Judge
LEARN · CODE · GROW

zjr284

管理员

题目

评测记录

语言管理

AI 命题

个人主页

用户管理

访问审计

退出登录

生成的题目：青蛙跳石板（1001）

难度 中等 · 时间限制 2.0s · 内存限制 128MB · 动态规划 单调队列

题目描述 样例 测试点

平静的湖面上有一排 n 片石板，从左至右编号为 1 到 n 。在每片石板 i 上有一个整数分数 s_i ，可能为负数。小青蛙一开始在岸上（位置 0），每次跳跃最多只能向前越过 m 片石板，也就是说，若当前位置为 i ，下一次可以落到 $i+1, i+2, \dots, i+m$ 中的任意一片石板上。青蛙必须最终恰好落到第 n 片石板上，落到某片石板时获得该石板的分数。请计算小青蛙最多能获得的总分数。

输入格式

一行三个整数 $n, m, seed$ ，表示石板数、一次最大跳跃跨度和随机种子，满足 $1 \leq m \leq n \leq 10^6, 0 \leq seed \leq 10^9$ 。石板分数的生成规则为：对 $i=1..n, s_i = ((seed * i) \% 2001) - 1000$ 。即先计算 $seed$ 乘以 i 后对 2001 取模，再减去 1000。

输出格式

输出一个整数，表示小青蛙最多能获得的总分数。

数据范围

$1 \leq m \leq n \leq 10^6, 0 \leq seed \leq 10^9$ 。分数 s_i 在 $[-1000, 1000]$ 内。

审阅并修改题目 JSON（保存时使用此内容）

```
{
  "id": "1001",
  "title": "青蛙跳石板",
  "description": "平静的湖面上有一排 n 片石板，从左至右编号为 1 到 n。在每片石板 i 上有一个整数分数 s_i，可能为负数。小青蛙一开始在岸上（位置 0），每次跳跃最多只能向前越过 m 片石板，也就是说，若当前位置为 i，下一次可以落到 i+1, i+2, ..., i+m 中的任意一片石板上。青蛙必须最终恰好落到第 n 片石板上，落到某片石板时获得该石板的分数。请计算小青蛙最多能获得的总分数。",
  "input_description": "一行三个整数 n, m, seed，表示石板数、一次最大跳跃跨度和随机种子，满足 1 ≤ m ≤ n ≤ 10^6, 0 ≤ seed ≤ 10^9。石板分数的生成规则为：对 i=1..n, s_i = ((seed * i) % 2001) - 1000。即先计算 seed 乘以 i 后对 2001 取模，再减去 1000。",
  "output_description": "输出一个整数，表示小青蛙最多能获得的总分数。",
  "samples": [
  ]
}
```



6. AI 使用说明

本项目开发过程中使用 Codex 辅助进行代码阅读、问题定位、实现建议、回归检查和本报告初稿整理；功能设计、需求取舍、代码审阅和最终演示由本人确认完成。

本人实际投入时间为大约24小时；AI 辅助代码/文本的估计占比为 95 %。

7. 总结

本实验完成了一个具备完整业务闭环的 OJ 系统。实现过程中最关键的体会是：判题状态、资源回收、会话恢复、权限边界和 AI 任务生命周期都属于跨模块问题，不能只依赖单一页面或单个接口验证。通过将耗时任务后台化、将权限集中到后端、将页面导航交给 Streamlit 原生机制，系统的稳定性和使用体验得到改善。

